

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Name: GAYATHRI SRIVALLI KAJULURI Roll No.: 2520080010 CO Mapping: CO2

SKILLING SESSION 8: VECTOR DATABASES – IMPLEMENTATION AND INTEGRATION

Aim

To store and query embeddings in PostgreSQL with pgvector (vector column and the <->, <=>, <#> operators), create HNSW and IVFFlat vector indexes, implement hybrid search that combines a keyword rank with vector similarity, outline the RAG retrieval flow, and compare with Weaviate and Pinecone.

1. PostgreSQL with pgvector

The implementation stores Task Hub help-centre articles in PostgreSQL using an 8-dimensional VECTOR column. Embeddings are generated when the articles are inserted. A generated TSVECTOR column is also created for keyword search.

Table definition:

```
CREATE TABLE help_articles (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    embedding VECTOR(8) NOT NULL,
    tsv TSVECTOR GENERATED ALWAYS AS
    (to_tsvector('english', title || ' ' || body)) STORED
)
```

The program inserts 12 help articles, each with an 8-dimensional embedding.

2. pgvector Similarity Operators

For the query “I cannot sign in, my login token stopped working”, the three pgvector operators are demonstrated: <-> for L2 distance, <=> for cosine distance, and <#> for negative inner product.

Title	<-> L2	<=> Cosine	-(<#>) Inner Product
Session token expired	0.2674	0.0358	0.9642
API rate limits	0.7532	0.2837	0.7163
Enable two-factor login	1.0353	0.5359	0.4641
Reset your password	1.1497	0.6608	0.3392

<#> returns the negative inner product, so the value is multiplied by -1 when displayed as an inner-product score.

3. Vector Indexes: HNSW and IVFFlat

The experiment creates 20,000 vectors and compares an unindexed search with HNSW and IVFFlat indexes. EXPLAIN shows how the execution plan changes from a sequential scan and sort to an index scan.

HNSW index:

```
CREATE INDEX idx_bench_hnsw
ON vector_bench USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

```
SET hnsw.ef_search = 40;
```

IVFFlat index:

```
CREATE INDEX idx_bench_ivf
ON vector_bench USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

```
SET ivfflat.probes = 10;
```

Search	Execution plan / result
Without index	Seq Scan on vector_bench → Sort → Limit
HNSW	Index Scan using idx_bench_hnsw

IVFFlat	Index Scan using idx_bench_ivf
IVFFlat recall@10	1.0 versus exact search

4. Hybrid Search

Hybrid search combines keyword relevance with vector similarity. The implementation uses PostgreSQL's **ts_rank_cd** as the keyword score and cosine-based vector similarity as the semantic score. The two scores are combined with equal 0.5/0.5 weights.

```
WITH keyword AS (  
  SELECT id,  
         ts_rank_cd(tsv, websearch_to_tsquery('english', %(text)s)) AS kw_score  
  FROM help_articles  
) , semantic AS (  
  SELECT id, 1 - (embedding <=> %(vec)s) AS vec_score  
  FROM help_articles  
)  
SELECT ... ORDER BY hybrid DESC LIMIT 3;
```

Title	Keyword	Vector	Hybrid (0.5/0.5)
Export tasks to CSV	0.075	0.970	0.985
Delete a project	0.000	0.375	0.188
Set task due dates	0.000	0.345	0.172

5. RAG Retrieval Flow

The Retrieval-Augmented Generation (RAG) step first retrieves relevant documents using vector similarity, builds a prompt containing the retrieved context, and then passes that prompt to an LLM. The example asks how to get back into an account after forgetting a password.

```
Retrieve context  
  ↓  
Build prompt using retrieved articles  
  ↓  
Send prompt + question to an LLM  
  ↓  
Generate answer using the supplied context
```

For the demonstrated question, the retrieved context included “Reset your password” and “Enable two-factor login” before the LLM prompt was constructed.

6. Weaviate and Pinecone Reference

The source includes equivalent reference client code for Weaviate and Pinecone. The Weaviate example uses a self-provided vector configuration, near-vector search, and hybrid search with BM25 plus vector similarity. The Pinecone example creates a cosine-metric index, upserts vectors with metadata, and queries the index with metadata filtering.

Feature	Weaviate	Pinecone
Deployment in source	Self-hosted	Managed cloud
Vector search	near_vector	query
Hybrid search	BM25 + vector, alpha=0.5	Reference code focuses on vector query + meta filter
Example filtering	—	category = account

Output / Result

Twelve help articles were stored with 8-dimensional embeddings. For the login query, all three pgvector operators ranked “Session token expired” first. EXPLAIN changed from a sequential scan plus sort to an index scan for both HNSW and IVFFlat, and IVFFlat returned the same top 10 as exact search with recall 1.0. Hybrid scoring placed “Export tasks to CSV” first. The RAG step retrieved the password-reset article as context for the LLM prompt. The source also provides reference implementations for Weaviate and Pinecone.